

main.py

```
1 import cv2
2 import io
3 import os
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from PIL import Image
7 from scipy.fftpack import dctn, idctn
8 from skimage.metrics import structural_similarity as ssim
9
10 BASE_QUANTIZATION_MATRIX = np.array([
11     [16,11,10,16,24,40,51,61],
12     [12,12,14,19,26,58,60,55],
13     [14,13,16,24,40,57,69,56],
14     [14,17,22,29,51,87,80,62],
15     [18,22,37,56,68,109,103,77],
16     [24,35,55,64,81,104,113,92],
17     [49,64,78,87,103,121,120,101],
18     [72,92,95,98,112,100,103,99]
19 ], dtype=np.float32)
20
21 def get_quantization_matrix(quality=50):
22     quality = np.clip(quality, 1, 100)
23     scale = (5000 / quality) if quality < 50 else (200 - 2 * quality)
24     return np.clip((BASE_QUANTIZATION_MATRIX * scale + 50) / 100, 1, 255)
25
26 def process_image_dct(image, quantization_matrix):
27     h, w = image.shape
28     img_float = np.float32(image) - 128
29
30     blocks = img_float.reshape(h // 8, 8, w // 8, 8).transpose(0, 2, 1, 3)
31
32     dct_blocks = dctn(blocks, axes=(-2, -1), norm='ortho')
33     quantized = np.round(dct_blocks / quantization_matrix)
34     dequantized = quantized * quantization_matrix
35     reconstruct = idctn(dequantized, axes=(-2, -1), norm='ortho')
36
37     result = reconstruct.transpose(0, 2, 1, 3).reshape(h, w) + 128
38
39     return np.clip(result, 0, 255).astype(np.uint8)
40
41 def visualize_dct_coefficients(image):
42     img_float = np.float32(image) - 128
43     blocks = img_float.reshape(image.shape[0] // 8, 8, image.shape[1] // 8,
44 8).transpose(0, 2, 1, 3)
45     dct_blocks = dctn(blocks, axes=(-2, -1), norm='ortho')
```

```
46 avg_coeff = np.mean(np.abs(dct_blocks), axis=(0,1))
47
48 plt.figure(figsize=(5, 4))
49 plt.imshow(np.log(avg_coeff + 1), cmap='hot')
50 plt.colorbar(label='log(|DCT coeff| + 1)')
51 plt.title("Average DCT Coefficient Magnitude\n(log scale – energy concentrates top-left)")
52 plt.xlabel("Horizontal frequency")
53 plt.ylabel("Vertical frequency")
54 plt.tight_layout()
55 plt.savefig("dct_coefficients.png", dpi=150)
56 plt.show()
57
58 def process_ycbcr(image_ycbcr, quality=50):
59     qm = get_quantization_matrix(quality)
60
61     chroma_qm = np.clip(qm * 2, 1, 255)
62
63     channels = cv2.split(image_ycbcr)
64     processed = [
65         process_image_dct(channels[0], qm),
66         process_image_dct(channels[1], chroma_qm),
67         process_image_dct(channels[2], chroma_qm),
68     ]
69     return cv2.merge(processed)
70
71 def compute_metrics(original, compressed):
72     mse = np.mean((original.astype(np.float32) - compressed.astype(np.float32))
73 ** 2)
74     psnr_db = min(20 * np.log10(255 / np.sqrt(mse)), 48.1)
75
76     psnr_percent = (psnr_db / 48.5) * 100
77     ssim_score = ssim(original, compressed)
78
79     return mse, psnr_percent, ssim_score
80
81 def compute_size(img_ycrb, quality=95):
82     img_bgr = cv2.cvtColor(img_ycrb, cv2.COLOR_YCrCb2BGR)
83     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
84     pil_img = Image.fromarray(img_rgb)
85     buf = io.BytesIO()
86     pil_img.save(buf, format='JPEG', quality=quality)
87     return buf.tell() / 1024
88
89 image = cv2.imread("images/cat.png", cv2.IMREAD_COLOR)
90 if image is None:
91     print("Image not found.")
92     exit()
```

```
92 image = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)
93
94 h, w = image.shape[:2]
95 image = image[:h - h % 8, :w - w % 8]
96 print(f"Image size: {image.shape}")
97
98 original_size = os.path.getsize("images/cat.png") / 1024
99 original_encoded = compute_size(image)
100 print(f"Image dimensions: {image.shape}")
101 print(f"Original Size: {original_size: .2f} KB")
102 print(f"Original Encoded Size: {original_encoded: .2f} KB")
103
104 quality_levels = [5, 25, 50, 75, 95]
105 results = []
106
107 for q in quality_levels:
108     comp = process_ycbcr(image, q)
109
110     comp_bgr = cv2.cvtColor(comp, cv2.COLOR_YCrCb2BGR)
111     orig_bgr = cv2.cvtColor(image, cv2.COLOR_YCrCb2BGR)
112
113     mse, psnr_percent, ssim_score = compute_metrics(
114         cv2.split(image)[0],
115         cv2.split(comp)[0]
116     )
117
118     mse, psnr_percent, ssim_score = compute_metrics(cv2.split(image)[0],
119 cv2.split(comp)[0])
120     comp_kb = compute_size(comp, quality=q)
121     ratio = original_size / comp_kb
122     results.append((q, comp, mse, psnr_percent, ssim_score, comp_kb, ratio))
123
124     print("\n")
125     print(f"Compressed size in KB: {comp_kb:3.2f}")
126     print(f"Quality Level: {q:3d}")
127     print(f"MSE: {mse:8.2f}")
128     print(f"PSNR: {psnr_percent:6.2f}")
129     print(f"SSIM: {ssim_score:4f}")
130
131 fig, axes = plt.subplots(2, 3, figsize=(15, 9))
132 axes = axes.flatten()
133
134 orig_bgr = cv2.cvtColor(image, cv2.COLOR_YCrCb2BGR)
135 axes[0].imshow(orig_bgr[:, :, ::-1])
136 axes[0].set_title(f"Original Image\n{original_size:.1f} KB", fontsize=13,
137 fontweight='bold')
138 axes[0].axis("off")
139
```

```

138 for idx, (q, comp, mse, psnr_percent, ssim_score, comp_kb, ratio) in
    enumerate(results, start=1):
139     comp_rgb = cv2.cvtColor(comp, cv2.COLOR_YCrCb2BGR)[:,:,:-1]
140     axes[idx].imshow(comp_rgb)
141     axes[idx].set_title(
142         f"Quality = {q} | Compressed size in KB: {comp_kb:.1f} KB \nCompressed
Ratio: {ratio:.1f}\nPSNR={psnr_percent:.1f} % | SSIM={ssim_score:.3f}",
143         fontsize=10
144     )
145     axes[idx].axis("off")
146
147 plt.suptitle("JPEG DCT Compression at Multiple Quality Levels", fontsize=14,
fontweight='bold')
148 plt.tight_layout()
149 # plt.savefig("compression_comparison.png", dpi=150)
150 plt.show()
151 # print(results)
152 fig, ax = plt.subplots(figsize=(9, 5))
153
154 labels = [f"Q={r[0]}" for r in results]
155 comp_sizes = [r[5] for r in results]
156 colors = ['#d73027', '#fc8d59', '#fee090', '#91cf60', '#1a9850']
157
158 bars = ax.bar(labels, comp_sizes, color=colors, edgecolor='black',
linewidth=0.8, zorder=3)
159 ax.axhline(original_size, color='steelblue', linestyle='--', linewidth=2,
160             label=f'Original ({original_size:.1f} KB)')
161
162 for bar, kb, ratio in zip(bars, comp_sizes, [r[6] for r in results]):
163     ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.5,
164             f"{kb:.1f} KB\n({ratio:.1f}x)", ha='center', va='bottom', fontsize=9,
fontweight='bold'
165             )
166 ax.set_xlabel("Quality Factor", fontsize=12)
167 ax.set_ylabel("File Size (KB)", fontsize=12)
168 ax.set_title("Compressed Image File Sizes vs Original", fontsize=13,
fontweight='bold')
169 ax.legend(fontsize=10)
170 ax.grid(axis='y', alpha=0.4, zorder=0)
171 ax.set_ylim(0, max(original_size, max(comp_sizes)) * 1.3)
172 plt.tight_layout()
173 # plt.savefig("size_comparison_bar.png", dpi=150)
174 plt.show()
175
176 qs = [r[0] for r in results]
177 psnrs = [r[3] for r in results]
178 ssims = [r[4] for r in results]
179

```

```
180 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
181
182 ax1.plot(qs, psnrs, 'bo-', linewidth=2, markersize=7)
183 ax1.set_xlabel("Quality Factor")
184 ax1.set_ylabel("PSNR(dB)")
185 ax1.set_title("PSNR vs Quality")
186 ax1.grid(True, alpha=0.4)
187
188 ax2.plot(qs, ssims, 'rs-', linewidth=2, markersize=7)
189 ax2.set_xlabel("Quality Factor")
190 ax2.set_ylabel("SSIM")
191 ax2.set_title("SSIM vs Quality")
192
193 ax2.grid(True, alpha=0.4)
194 plt.suptitle("Image Quality Metrics vs Compression Quality Factor",
195             fontsize=13)
196 plt.tight_layout()
197 # plt.savefig("metrics_vs_quality.png", dpi=150)
198 plt.show()
199
200 visualize_dct_coefficients(cv2.split(image)[0])
```